SmartPOS

Project Guide

Point-of-Sale and Inventory Management System

Directory structure, every module and file, how and why it was built, what is done and what is left.

Stack: Angular 22 · ASP.NET Core 10 · Entity Framework Core 10 · MySQL 8+ · JWT
Status: working foundation, verified end to end (October 2026)

1. What SmartPOS is

SmartPOS is a retail point-of-sale (POS) and inventory system. A cashier searches products, builds a cart, applies a permitted discount and takes payment; the system prices the sale, reduces stock, records every stock change and prints an invoice. An administrator manages products, stock, users and reports.

This repository is the **foundation** the team will extend: everything below is real, connected code (no fake statistics), but it is deliberately small. Offline billing and AI features were explicitly out of scope.

Technology choices and why

Technology	Role and reason
ASP.NET Core 10 Web API	The only .NET SDK installed on the machine is 10, so every package had to support .NET 10. Controllers stay thin; logic lives in services so it can be unit-tested without HTTP.
EF Core 10 + MySql.EntityFrameworkCore 10.0.12	EF Core gives migrations (versioned schema changes) and LINQ queries. The popular Pomelo MySQL provider has no EF Core 10 release yet, so Oracle's official provider was used. Trade-off: it does not support MariaDB (XAMPP's database).
MySQL 8.0.16+	Required by the brief. 8.0.16 is the first version that enforces CHECK constraints, which we use as a last line of defence against negative stock.
JWT bearer tokens	Stateless login that works naturally with an Angular single-page app. The token carries the user id and role.
ASP.NET Identity PasswordHasher	A recognised, maintained PBKDF2 implementation with a per-password salt and automatic re-hash support. We use only the hasher, not the whole Identity user store.
Angular 22 (standalone, signals, zoneless)	Current Angular conventions: no NgModules, signals for state, OnPush change detection everywhere, lazy-loaded routes.
Plain CSS design system	One stylesheet of design tokens (colours, radii, spacing) and shared classes. No UI framework dependency, so the look is fully ours: charcoal sidebar, green accent, white/grey surfaces.
xUnit, SQLite, Vitest	xUnit for backend tests; in-memory SQLite lets API integration tests run without a database server; Vitest ships with Angular 22.

How to read this guide

- Section 2 is the story of what was built, in order.
- Section 3 is the directory tree. Sections 4-7 explain each project and file, with the key code and the reasoning.
- Section 8 follows two complete flows (login, checkout) through every layer.
- Section 9 lists problems found along the way; Section 10 states what was verified; Section 11 lists what is left.

2. What we did, in order

Phase	What happened
0. Inspect	Checked OS (macOS, Apple Silicon), .NET SDK (10.0 only), Node 24, npm, Homebrew MySQL 9.6 (stopped), XAMPP MariaDB on port 3306. Looked up which NuGet packages support .NET 10. Found the folder already held old FinSight files (deleted in the working tree), so SmartPOS was created in a new `smartpos/` folder and the old files were left alone.
1. Scaffold	Created the solution (SmartPOS.slnx) with four layer projects and two test projects, central package versions, and the Angular 22 app with environments.
2. Database	Wrote six entities and their EF Core configurations (keys, indexes, decimal(18,2) money, CHECK constraints), generated the InitialCreate migration, and produced database/schema.sql from it.
3. Core API	Implemented JWT login, role authorization, and services plus controllers for products, categories, inventory, sales, reports, dashboard, users, settings. Added global error handling, health check, Swagger, CORS, login rate limiting.
4-5. Frontend	Built the shell (sidebar/header), login, dashboard, products, inventory, POS, sales/invoice, reports and users screens, wired to the real API with validation, loading, error and empty states, confirmation dialogs.
6. Tests + docs	Wrote 38 unit, 25 integration and 14 frontend tests; wrote README, ARCHITECTURE, DATABASE, API, TESTING docs and this guide.
7. Verify	Started a throwaway MySQL 9.6, applied the migration with `dotnet ef database update`, ran the API, ran a 48-check HTTP smoke test (including a concurrency test), and a 21-check headless-Chrome UI walkthrough. Stopped and discarded the temporary database afterwards.

3. Directory structure

```
smartpos/
|-- README.md                 Quick start, configuration table, features, limitations
|-- .gitignore                Keeps secrets, bin/obj, node_modules, dist out of git
|-- .env.example              Names of required environment variables (no real values)
|-- database/
|   `-- schema.sql            SQL generated from the EF Core migration (reference only)
|-- docs/
|   |-- ARCHITECTURE.md  DATABASE.md  API.md  TESTING.md
|   `-- SmartPOS-Project-Guide.pdf   (this document)
|-- backend/
|   |-- SmartPOS.slnx                     solution file
|   |-- Directory.Build.props             shared compiler settings (net10.0, nullable, warnings as errors)
|   |-- Directory.Packages.props          ONE place for every NuGet version
|   |-- .config/dotnet-tools.json         pins the dotnet-ef tool to 10.0.12
|   |-- src/
|   |   |-- SmartPOS.Core/                entities + enums (no dependencies)
|   |   |-- SmartPOS.Application/         services, DTOs, business rules, interfaces
|   |   |-- SmartPOS.Infrastructure/      EF Core, MySQL, migrations, password hashing, seeding
|   |   `-- SmartPOS.API/                 controllers, JWT, middleware, Program.cs
|   `-- tests/
|       |-- SmartPOS.UnitTests/           pure logic tests
|       `-- SmartPOS.IntegrationTests/    full HTTP pipeline on in-memory SQLite
`-- frontend/
    |-- angular.json  package.json  tsconfig*.json
    `-- src/
        |-- index.html  main.ts  styles.css         page shell + design system
        |-- environments/                           API URL per build
        `-- app/
            |-- app.ts  app.config.ts  app.routes.ts
            |-- core/       auth, API services, models, helpers, toast/confirm
            |-- shared/     reusable components and the money pipe
            |-- layout/     shell, sidebar, header
            `-- features/   auth, dashboard, products, inventory, pos, sales, reports, users
```

Dependency direction (the most important rule)

```
SmartPOS.API  ->  SmartPOS.Infrastructure  ->  SmartPOS.Application  ->  SmartPOS.Core
```

Arrows mean "may reference". Core knows nothing about anything else. Application defines *what* the system does and the interfaces it needs, but not *how* the database or HTTP work. Infrastructure supplies the database. API is the outermost shell. Because dependencies only point inwards, business logic can be tested without a web server or a database.

4. Backend: solution-level files and SmartPOS.Core

Solution-level files

File	What it does / why it exists
`Directory.Build.props`	Applied automatically to every project: net10.0, nullable reference types on, implicit usings, **TreatWarningsAsErrors**. Why: catches null-reference mistakes at compile time and keeps every project configured identically.
`Directory.Packages.props`	Central package management: every NuGet version is declared once; csproj files list package names only. Why: no version drift between projects. EF Core is pinned to 10.0.12 to match the MySQL provider.
`.config/dotnet-tools.json`	Pins `dotnet-ef` 10.0.12 as a local tool. Why: every developer runs the same migration tool via `dotnet tool restore`; nothing global to install.
`SmartPOS.slnx`	Solution listing the six projects under src/ and tests/ folders.

SmartPOS.Core: the domain

Plain C# classes and enums with no framework dependencies. They describe *what data exists*.

File	What it does / why it exists
Entities/User.cs	Id, FullName, Email (stored lower-case so uniqueness is case-insensitive), PasswordHash, Role, IsActive, timestamps, LastLoginAt. Only the hash is stored; the plaintext password never exists in the database.
Entities/Category.cs	Name (unique) and optional Description; owns a collection of Products.
Entities/Product.cs	Sku, Name, Description, CategoryId, SellingPrice, CostPrice, StockQuantity, ReorderLevel, IsActive, timestamps. `IsLowStock` is computed (stock <= reorder level). Why IsActive: a sold product cannot be deleted, so it is deactivated instead and disappears from the POS.
Entities/Sale.cs	InvoiceNumber, CashierId, Subtotal, DiscountPercent/Amount, **TaxRatePercent**/TaxAmount, Total, PaymentMethod, CreatedAt, Items. The tax rate in force is stored so an old invoice can always be explained even if the rate changes.
Entities/SaleItem.cs	A line on a sale: ProductId plus **copies** of ProductName, Sku, UnitPrice and UnitCost taken at sale time. This is how historical prices are preserved when a product's price later changes.
Entities/StockMovement.cs	Append-only ledger of every stock change: QuantityChange (+/-), QuantityAfter, Type, Reason, optional SaleId and UserId. Why: stock is never changed silently, so any number can be audited.
Enums/UserRole.cs	Admin, Cashier.
Enums/PaymentMethod.cs	Cash, Card, Other.
Enums/StockMovementType.cs	InitialStock, Sale, Adjustment.

5. SmartPOS.Application: services and business rules

This is where the system's behaviour lives. Each feature area has a service class behind an interface (so controllers and tests depend on the interface), DTO classes that define request and response shapes with validation attributes, and small pure-logic classes that can be tested in isolation.

Common/ (shared building blocks)

File	What it does / why it exists
AppExceptions.cs	BusinessRuleException (400), AuthenticationFailedException (401), ForbiddenException (403), NotFoundException (404), ConflictException (409). Services throw these; one global handler in the API converts them to HTTP responses. Why: services never reference HTTP, and error handling is not repeated in every controller.
IApplicationDbContext.cs	The persistence surface services use: the six DbSets, SaveChanges, BeginTransaction, and `TryChangeStockAsync`. Why not one repository per entity: EF Core already is a repository/unit-of-work; wrapping it adds files without adding safety. The one extra method needs special care, so it is centralised.
ICurrentUser.cs	UserId and Role of the caller. The API implements it from JWT claims; services use it for 'who made this sale/adjustment' and role-based rules.
Paging.cs	PagedQuery (page 1+, pageSize 1-100), PagedResult<T> (items, page, pageSize, totalCount, totalPages) and a ToPagedResultAsync extension used by every list endpoint.
SearchPattern.cs	Builds SQL LIKE patterns, escaping user-typed % and _ so a search for '50%' matches literally. Uses '!' as the escape character because the MySQL provider's default backslash produced invalid SQL (found during MySQL testing).
Options.cs	StoreOptions (name, currency, time zone) and PosOptions (tax rate, max discount per role) bound from configuration and validated at startup.
StoreCalendar.cs	Converts between UTC (how timestamps are stored) and the store's local calendar days (how people ask: 'today'). Why: a shop in India asking for 'today' must mean India's midnight-to-midnight, wherever the server runs.
MaxDecimalPlacesAttribute.cs	Validation attribute rejecting money with more than 2 decimals, matching the database column.

Sales/: the heart of the system

File	What it does / why it exists
SaleCalculator.cs	Pure function: line totals, subtotal, discount, tax, total. Rule order: discount is taken off the subtotal first, then tax is charged on the discounted amount. Rounds to 2 decimals, half away from zero. Rejects empty sales, zero/negative quantities, negative prices, out-of-range percentages.
DiscountPolicy.cs	Maximum discount per role (default Cashier 10%, Admin 100%). Throws ForbiddenException above the limit. Why server-side: a cashier editing the browser cannot exceed their authority.
SaleDtos.cs	CreateSaleRequest contains only product ids, quantities, discount % and payment method. **No prices or totals**: the client cannot choose what it pays. Also SaleDto (invoice), SaleSummaryDto (list row), SaleQuery, PosSettingsDto.
SaleService.cs	Orchestrates checkout (walkthrough in Section 8). Also lists sales (cashiers see only their own) and loads one sale (another cashier's sale returns 404 so ids cannot be probed).
ISaleService.cs	Interface for the above.

```
// SaleCalculator.Calculate (simplified)
```

```
var subtotal = lineTotals.Sum();
var discount = Round(subtotal * discountPercent / 100m);
var tax      = Round((subtotal - discount) * taxRatePercent / 100m);
return new SaleTotals(lineTotals, subtotal, discount, tax, subtotal - discount + tax);
// Example: 3 x 99.99 + 3 x 40 = 419.97; 10% off = 42.00; 5% tax on 377.97 = 18.90; total 396.87
```

Inventory/, Products/, Categories/

File	What it does / why it exists
Inventory/StockRules.cs	EnsureAvailable (requested <= on hand, with a friendly message), EnsureValidAdjustment (non-zero, cannot go below 0), GetStatus (InStock / Low / OutOfStock). 'Low' means at or below reorder level, the same definition the dashboard uses.
Inventory/InventoryService.cs	Lists stock sorted most-urgent first, records adjustments (inside a transaction, using TryChangeStockAsync), lists the movement ledger with filters.
Inventory/InventoryDtos.cs, IInventoryService.cs	Request/response shapes and interface. A stock adjustment needs a reason of at least 3 characters.
Products/ProductService.cs	List/search (name or SKU), create, update, delete. SKU is upper-cased and checked for duplicates (friendly pre-check; the unique index is the real guarantee). Create records opening stock as a movement. Update **cannot change stock**. Delete refuses products that appear on any sale.
Products/ProductDtos.cs	Validation: SKU pattern and length, prices 0 to 9,999,999.99 with at most 2 decimals, reorder level and opening stock 0 to 1,000,000.
Categories/Categories.cs	DTOs, interface and service in one file (small feature). Case-insensitive duplicate check.

Auth/, Users/, Reports/, Dashboard/

File	What it does / why it exists
Auth/AuthService.cs	Login: normalise email, find user, verify hash. Unknown email, wrong password and deactivated account all return the same 'Invalid email or password' (no account enumeration); unknown emails still do a dummy hash check so timing does not leak existence. Upgrades old hashes transparently; records last login.
Auth/SecurityAbstractions.cs	IPasswordHasher and ITokenService interfaces (implemented in Infrastructure and API).
Auth/AuthDtos.cs	LoginRequest, LoginResponse (accessToken, expiresAt, user).
Users/UserService.cs, UserDtos.cs	Create users (password rule: 8-128 chars, one letter, one digit; unique email), update name/role/active/password. An admin cannot change their own role or deactivate themselves, to prevent locking everyone out.
Reports/Reports.cs	Daily sales (every day in range, including empty days) and top products by revenue with cost and gross profit. Max range 366 days. Reads only recorded sales and sale-item snapshots.
Dashboard/Dashboard.cs	One call returns today's total/count (store time zone), active product count, low-stock count, 8 recent sales, 8 lowest-stock products. Empty database gives zeros.
DependencyInjection.cs	AddApplication(): registers options (validated at startup), TimeProvider, StoreCalendar and every service.

6. SmartPOS.Infrastructure: database and security

File	What it does / why it exists
Persistence/SmartPosDbContext.cs	The EF Core context. Applies all configuration classes, marks every DateTime as UTC when read back (MySQL DATETIME has no zone), and implements `TryChangeStockAsync` (below).
Persistence/Configurations/*.cs	One class per entity: max lengths, **decimal(18,2)** money, unique indexes (Sku, Email, Category name, InvoiceNumber), CHECK constraints (stock >= 0, prices >= 0, quantity > 0), foreign keys. Delete rules: Restrict where history must survive (Sale->User, SaleItem->Product), Cascade where it should follow (SaleItems with their Sale).
Persistence/Migrations/	Generated by `dotnet ef migrations add InitialCreate`: the versioned CREATE TABLE script plus a snapshot used to compute future changes. Primary schema mechanism.
Persistence/DatabaseInitializer.cs	Runs at startup. (a) If Bootstrap admin settings exist and no admin exists, creates the first admin. (b) If Database:SeedSampleData is true and the catalogue is empty, inserts sample data. Skips with a warning if the database is unreachable or migrations are pending; never creates or alters the schema itself.
Persistence/SampleData.cs	6 categories and 16 fictional products, some deliberately below reorder level so low-stock screens have content. No users, no sales.
Security/IdentityPasswordHasher.cs	Adapter over ASP.NET Identity's PasswordHasher: PBKDF2-HMAC-SHA512, random salt per password.
DependencyInjection.cs	AddInfrastructure(): registers the DbContext with `UseMySQL(connectionString)`, reads the connection string lazily with a clear error if missing.

The anti-overselling primitive

```
// SmartPosDbContext.TryChangeStockAsync
var affected = await Products
    .Where(p => p.Id == productId && p.StockQuantity >= -delta)
    .ExecuteUpdateAsync(s => s
        .SetProperty(p => p.StockQuantity, p => p.StockQuantity + delta)
        .SetProperty(p => p.UpdatedAt, now), ct);
if (affected == 0) return null;   // not enough stock (or no such product)
```

Why: This becomes a single SQL statement: `UPDATE Products SET StockQuantity = StockQuantity + @d WHERE Id = @id AND StockQuantity >= @need`. The check and the change cannot be separated, and InnoDB keeps the row locked until the surrounding transaction commits. Two cashiers selling the last unit at once: one UPDATE matches, the other matches zero rows and the sale is rejected. Reading stock into memory, checking it, then writing it back would allow both to succeed.

7. SmartPOS.API: the HTTP layer

File	What it does / why it exists
Program.cs	Startup. Order matters: exception handler, status-code pages, Swagger (Development only) or HSTS, CORS, authentication, authorization, rate limiter, controllers, /health, then the database initializer.
Infrastructure/ApiServiceCollectionExtensions.cs	Wires JWT bearer validation (issuer, audience, signing key, lifetime, 30 s skew), the **fallback authorization policy** (every endpoint requires login unless marked AllowAnonymous), Swagger with a Bearer button, CORS for configured origins, login rate limit, health check. Also the OnTokenValidated check: a valid token is rejected if the user has since been deactivated or had their role changed.
Infrastructure/GlobalExceptionHandler.cs	Maps exceptions to RFC 7807 problem responses. Expected exceptions return their message; unexpected ones return a generic 500 with a trace id while the full exception goes only to the server log.
Infrastructure/DatabaseHealthCheck.cs	/health reports Healthy/Unhealthy without exposing exception text (it can contain host names).
Auth/JwtOptions.cs	Validated configuration. The signing key must be present and at least 32 characters or the API refuses to start.
Auth/JwtTokenService.cs	Creates the token (claims: sub, name, email, role, jti) signed with HMAC-SHA256.
Auth/CurrentUser.cs	Reads user id and role from the JWT claims for the services; also defines the role-name constants used in [Authorize].
Controllers/*.cs	Auth, Dashboard, Products, Categories, Inventory, Sales, Reports, Users, Settings. Each action is one line delegating to a service. Role rules are attributes: `[Authorize(Roles = Roles.Admin)]` on create/update/delete product, adjustments, reports and users.
appsettings.json / .Development.json	Non-secret defaults only. ConnectionStrings:SmartPos, Jwt:SigningKey are empty placeholders; real values come from user-secrets or environment variables. Development enables CORS for localhost:4200 and sample data.
Properties/launchSettings.json	Dev profile: http://localhost:5080.
SmartPOS.API.http	Ready-made requests for VS Code / JetBrains HTTP clients.

Secrets: how they stay out of the code

- **Development:** `dotnet user-secrets` stores the connection string, signing key and first-admin credentials outside the repo.
- **Production:** environment variables (`ConnectionStrings__SmartPos`, `Jwt__SigningKey`); see .env.example.
- The first administrator is created from configuration, so there is no default password anywhere in code or migrations. Remove the bootstrap password after first start.

Database tables at a glance

Table	Key rules
Users	Unique Email. PasswordHash. Role as text.
Categories	Unique Name.
Products	Unique Sku; CHECK stock/prices >= 0; FK to Categories (restrict).
Sales	Unique InvoiceNumber; index on CreatedAt for date filters; FK to Users (restrict).
SaleItems	Snapshot columns; CHECK quantity > 0; FK to Sales (cascade) and Products (restrict).
StockMovements	Index (ProductId, CreatedAt); optional FK to Sales and Users.

All timestamps are `datetime(6)` in UTC. The store's time zone setting is applied only when interpreting 'today' and date filters.

Backend tests

File	What it does / why it exists
UnitTests/SaleCalculatorTests.cs	Totals (the worked example above), discount-before-tax order, half-away-from-zero rounding, invalid input.
UnitTests/StockRulesTests.cs	Selling the last unit is allowed, over-requesting is a conflict, adjustments cannot go negative, status boundaries.
UnitTests/DiscountPolicyTests.cs, StoreCalendarTests.cs, SearchPatternTests.cs	Role limits; 01:30 IST on 2 March still being 2 March; LIKE escaping and 2-decimal validation.
IntegrationTests/SmartPosApiFactory.cs	Boots the real API in memory, swaps MySQL for SQLite, seeds a test admin, cashier and category. (Test-only credentials in a throwaway database.)
IntegrationTests/AuthAndAuthorizationTests.cs	Login success, same error for wrong password and unknown user, 401 without token, 403 for cashiers on admin endpoints, admin creates cashier.
IntegrationTests/ProductAndSaleTests.cs	Duplicate SKU, validation errors, pagination, sale pricing and stock deduction with movements, oversell rolls back *every* line, discount limit, price history preserved, delete rules, adjustments, cashier visibility, dashboard figures.

8. Frontend (Angular 22)

Configuration and entry files

File	What it does / why it exists
environments/environment*.ts	apiUrl: development points to http://localhost:5080/api; production uses '/api' (same origin behind a proxy). angular.json swaps the file for development builds.
app.config.ts	Providers: router (with component input binding so route params arrive as inputs), HttpClient with the auth interceptor, custom title strategy ('Page - SmartPOS').
app.routes.ts	Every feature is lazy-loaded (loaded on first visit), so the login page downloads little. The authenticated area is a Shell route guarded by authGuard, which also pre-loads settings. Reports, Users, product create/edit use adminGuard.
styles.css	The design system: CSS variables (colours, radii, shadows), base elements, and shared classes: .card, .table, .btn (variants), .input, .badge, .alert, modal, responsive breakpoints, print rules.

core/: logic every screen shares

File	What it does / why it exists
models.ts	TypeScript mirrors of the API's DTOs, so a typo in a field name fails at compile time.
auth/auth.service.ts	Holds the session in a signal (user, isAdmin, isAuthenticated), persists it to localStorage, logs out automatically when the token expires.
auth/auth.interceptor.ts	Adds 'Authorization: Bearer ...' to every API call and logs out on a 401 from an authenticated call (expired token or deactivated user).
auth/auth.guards.ts	authGuard (must be signed in), guestGuard (signed-in users skip login), adminGuard. These improve navigation only: the API enforces every permission itself.
api/*.service.ts	Thin typed wrappers around HttpClient: products/categories, inventory, sales/reports/dashboard, users, settings (store name, currency, tax, max discount).
http/http-utils.ts	toHttpParams (skips empty values), errorMessage (turns the API's validation errors or detail into one readable sentence; explains network failures), date helpers.
ui/toast.service.ts, confirm.service.ts	Pop-up notifications and a promise-based confirmation: `if (await confirm.ask({...}))`.

shared/ and layout/

File	What it does / why it exists
components/icon.ts	Inline SVG icon set (no icon library or network request).
components/modal.ts, confirm-dialog.ts	Built on the native <dialog> element, which provides focus trapping and Escape-to-close for free (accessibility).
components/states.ts	EmptyState, Loading, ErrorAlert (with Try again). Every list shows one of these instead of a blank screen.
components/stat-card.ts, badges.ts, pagination.ts, toast-container.ts	Dashboard cards, stock/movement badges, pager, toasts.
pipes/money.pipe.ts	Formats amounts in the store's currency from settings (so INR shows the rupee sign).
layout/shell.ts, sidebar.ts, header.ts	Dark charcoal sidebar with active states, role-filtered menu, header with user and sign-out; becomes a slide-in menu under 900 px; skip-to-content link.

features/: one folder per screen

File	What it does / why it exists
auth/login	Email/password form with validation, show-password toggle, generic error. Return URL is accepted only if it is an in-app path (prevents open redirects).
dashboard	Four stat cards plus recent-transactions and low-stock tables, all from /api/dashboard/summary, with empty states.
products	product-list: debounced search, category filter, pagination, delete with confirmation. product-form: create/edit with validators matching the API, inline 'new category', stock shown read-only on edit (change it in Inventory).
inventory	inventory: stock levels, All/Low/Out filter, Adjust dialog showing the resulting level before saving. movements: filterable ledger.
pos	Fast checkout: product tiles, Enter-to-add for scanner/SKU input, cart with quantity clamped to stock, discount field limited to the role's maximum, payment method, live totals, 'Sale completed' panel with print link.
pos/cart.ts	Pure cart arithmetic in **integer cents and basis points**, mirroring the server's rounding exactly. Why: JavaScript decimals drift (19.99 x 3 = 59.97000000000001); cents avoid it so the preview equals the invoice.
sales	sales-list: date presets and invoice search. sale-detail: itemised printable invoice; print CSS hides the app chrome.
reports	Date range with presets: totals, average sale, per-day table with inline bars, top products with profit.
users	User table and a create/edit dialog; role and active controls are locked for your own account.

```
// pos/cart.ts - works in whole cents so results match the server to the cent
export function calculateTotals(lines, discountPercent, taxRatePercent) {
  const subtotalCents = lines.reduce((s, l) => s + toCents(l.unitPrice) * l.quantity, 0);
  const discountCents = roundDiv(subtotalCents * toBasisPoints(discountPercent), 10_000);
  const taxableCents  = subtotalCents - discountCents;
  const taxCents      = roundDiv(taxableCents * toBasisPoints(taxRatePercent), 10_000);
  return { subtotal: subtotalCents / 100, discountAmount: discountCents / 100,
           taxAmount: taxCents / 100, total: (taxableCents + taxCents) / 100 };
}
```

9. Two flows, end to end

A. Logging in

- Browser: the Login component posts email and password to `/api/auth/login`.
- API: rate limiter (10 attempts/min per IP) -> AuthController -> AuthService. It finds the user by lower-cased email and verifies the PBKDF2 hash.
- On success JwtTokenService signs a token (8 hours by default) with the user's id and role; the response includes the user profile.
- Browser: AuthService stores the session; the interceptor attaches the token to every later request; the router goes to /dashboard.
- Every later request: the API validates signature and expiry, then looks the user up again. A deactivated user's token stops working immediately.

B. Completing a sale (POST /api/sales)

- **Validate** the request shape (1-200 items, quantity 1-10,000, discount 0-100). Invalid -> 400 with field messages.
- **Authorize the discount** for the caller's role. Too high -> 403.
- **Merge duplicate lines** and sort by product id (a fixed locking order prevents two multi-item sales from deadlocking).
- **Load products and their prices from the database.** Missing or inactive -> 400. Not enough stock -> 409 with 'requested 6, available 5'.
- **Calculate** totals with SaleCalculator.
- **Open a transaction.** For each line run the conditional stock UPDATE. If any returns no row, leave the transaction without committing; MySQL rolls back the earlier lines.
- **Insert** the Sale, SaleItems (with price/name snapshots) and one StockMovement per line; **commit**.
- Return 201 with the full invoice. The browser shows the completed-sale panel and refreshes stock.

```
await using var transaction = await db.BeginTransactionAsync(ct);
foreach (var (productId, quantity) in lines)
{
    var quantityAfter = await db.TryChangeStockAsync(productId, -quantity, ct)
        ?? throw new ConflictException("Insufficient stock ... refresh and try again.");
    sale.Items.Add(new SaleItem { ProductId = productId, UnitPrice = product.SellingPrice, ... });
    db.StockMovements.Add(new StockMovement { QuantityChange = -quantity, QuantityAfter = quantityAfter, ... });
}
db.Sales.Add(sale);
await db.SaveChangesAsync(ct);
await transaction.CommitAsync(ct);   // nothing is permanent until this line
```

10. Problems found and how they were fixed

Problem	Resolution
No EF Core 10 build of the Pomelo MySQL provider	Used Oracle's MySql.EntityFrameworkCore 10.0.12 and pinned EF Core to the matching 10.0.12. Consequence: MariaDB (XAMPP) is unsupported.
Port 3306 is XAMPP's MariaDB, not MySQL	Did not use it. Ran a temporary MySQL 9.6 on port 3307 for testing and removed it afterwards. DATABASE.md explains how to run both.
Product search crashed on real MySQL	The provider emitted `ESCAPE '\'`, invalid in MySQL. Switched the escape character to '!'. In-memory SQLite tests could not have caught this; only the real-database run did.
Generated 'idempotent' migration script was invalid MySQL	Documented that it should not be used; `database/schema.sql` is the plain script, and `dotnet ef database update` is the supported path.
MySQL socket path too long on macOS	Test-only issue (103-character limit); fixed by using TCP and a short socket name.
A test helper produced a 3-decimal cost (99.99 / 2)	The validator correctly rejected it. Fixed the helper to round to 2 decimals.
A JavaScript rounding assertion was wrong (1.005 x 100)	Replaced with a realistic float-drift case (19.99 x 3).
UI test failed on re-run	The reused Chrome profile still held a signed-in session, so the login redirect correctly did not happen. Cleared the profile.

11. What was verified

Check	Result
Backend build (Release)	0 warnings, 0 errors
Backend unit tests	38 passed
Backend integration tests	25 passed (SQLite)
Frontend unit tests	14 passed
Frontend production build	Clean; initial bundle 312 kB (87 kB gzipped)
Migration on real MySQL 9.6	Applied; six tables, indexes and CHECK constraints created
API smoke test on MySQL	48 checks passed: roles, validation, arithmetic, rollback, price history, token revocation
Concurrency test	12 parallel sales for 5 units: exactly 5 succeeded, 7 rejected, final stock 0
Browser walkthrough (headless Chrome)	21 checks passed with no console errors or failed requests, including mobile layout and cashier restrictions
Secrets in logs	Checked: password, signing key and database password do not appear in the server log

Not verified: running against your own MySQL install, Windows or Linux, browsers other than Chrome, and the physical print output (the print dialog opens, but no printer was used).

12. What is left to do

Needed to start real use

- Install or start a real MySQL 8+ on your machine (not XAMPP's MariaDB), create the database and user, set user-secrets, run the migration.
- Choose real store settings: name, currency code, time zone and tax rate (defaults are INR, Asia/Kolkata, 5%).
- Production deployment: HTTPS, a reverse proxy serving the Angular build and forwarding /api, environment variables for secrets, a least-privilege database user, backups.

Features not built yet

- **Offline billing** (explicitly deferred). Plan: idempotency key on each sale, a client-side queue (IndexedDB plus service worker), replay on reconnect; the server's re-pricing and stock checks already make replays safe to accept or reject.
- Refunds, returns and voiding a sale (stock movements already allow new types).
- Per-line discounts, promotions, multiple tax rates, multiple stores or branches.
- Suppliers and purchase orders; barcode printing; hardware scanner and receipt-printer integration beyond typing a SKU.
- Product images, CSV import/export, charts on reports, audit log screen.
- Customer records and loyalty.

Engineering improvements

- Add a CI pipeline that runs both test suites against a MySQL service container (SQLite cannot reproduce MySQL's SQL dialect or locking).
- Move the login token from localStorage to an HTTP-only cookie or refresh-token flow for stronger protection against script injection.
- Add Angular component tests and Playwright end-to-end tests (the headless-Chrome check was a one-off script, not a committed test).
- Pre-aggregate daily totals if a store grows large (reports currently group in memory for the chosen range, maximum 366 days).
- Password reset and account lock-out flows; optional two-factor sign-in for admins.
- Structured logging sink and monitoring; Docker files if the team adopts containers later.

Run commands (reference)

```
export DOTNET_ROOT=$HOME/.dotnet PATH=$HOME/.dotnet:$PATH       # .NET is not on PATH on this machine

# one-time
cd smartpos/backend/src/SmartPOS.API
dotnet user-secrets set "ConnectionStrings:SmartPos" "Server=127.0.0.1;Port=3306;Database=smartpos_dev;User=smartpos;
dotnet user-secrets set "Jwt:SigningKey" "$(openssl rand -base64 48)"
dotnet user-secrets set "Bootstrap:AdminEmail" "you@example.com"
dotnet user-secrets set "Bootstrap:AdminPassword" "<8+ chars with a letter and a digit>"
cd ../.. && dotnet tool restore
dotnet ef database update --project src/SmartPOS.Infrastructure --startup-project src/SmartPOS.API

# every day
cd smartpos/backend  && dotnet run --project src/SmartPOS.API --launch-profile http   # API + /swagger
cd smartpos/frontend && npm install && npm start                                      # http://localhost:4200

# tests
cd smartpos/backend  && dotnet test SmartPOS.slnx
cd smartpos/frontend && npm test
```